Lab 8. Deploy to Dev with Docker Compose

By the end of this lab exercise, you should be able to:

- Explain how Docker Compose can launch a monitoring stack
- Add services to Docker Compose
- Create a consolidated Jenkins pipeline
- Start deploying to an integrated dev environment with Docker Compose

Before you begin, create a new branch and switch to it:

```
git checkout -b feature/monopipe
```

Also, enter the `devops-repo/setup` directory and shut Jenkins down while we explore the next steps:

```
cd devops-repo/setup
```

```
docker compose down
```

We will restart Jenkins after exploring the example voting application. If we do not shut Jenkins down, there will be port conflicts.

Create a Simple Docker Compose Spec

***NOTE**: Be sure to replace **xxxxx** with your Docker Hub ID.*

```
cd example-voting-app
```

Create the `docker-compose.yaml` with the following contents.

file: example-voting-app/docker-compose.yaml

```
services:
  vote:
    image: xxxxx/vote:latest
    ports:
      - 5000:80

  result:
    image: xxxxx/result:latest
    ports:
      - 5001:80

  worker:
    image: xxxxx/worker:latest
```

You can access the file here.

***NOTE**: Be sure to replace **xxxxx** with your Docker Hub ID.*

Launch the stack with:

```
docker compose up -d
```

```
docker compose ps
```

At this time, if you try submitting the vote from the UI, you will see an error. That is because you have not provisioned the backend service, i.e. **redis**.

Add **redis** along with **db** in docker-compose.yaml:

file: example-voting-app/docker-compose.yaml

```
services:
  vote:
    image: xxxxx/vote:latest
    ports:
      - 5000:80

  redis:
    image: redis:alpine

  db:
    image: postgres:15-alpine

  result:
    image: xxxxx/result:latest
    ports:
```

```
      - 5001:80

  worker:
    image: xxxxx/worker:latest
```

You can access this Docker Compose yaml file here.

***NOTE**: Be sure to replace **xxxxx** with your Docker Hub ID.*

Launch the new services using the same command as earlier:

```
docker compose up -d
```

```
docker compose ps
```

At this time, if you attempt to submit the vote using the UI, it should go through. This validates the service discovery (connecting **vote** with **redis**).

Add Networks and Volumes

To add networks and volumes to the example voting application, copy the following into your **example-voting-app/docker-compose.yaml**:

```
volumes:
  db-data:

networks:
  instavote:
    driver: bridge

services:
  vote:
    image: xxxxx/vote:latest
    ports:
      - 5000:80
    depends_on:
      - redis
    networks:
      - instavote

  redis:
    image: redis:alpine
    networks:
      - instavote

  db:
```

```
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: "postgres"
      POSTGRES_PASSWORD: "postgres"
    volumes:
      - "db-data:/var/lib/postgresql/data"
      - "./healthchecks:/healthchecks"
    healthcheck:
      test: /healthchecks/postgres.sh
      interval: "5s"
    networks:
      - instavote

  result:
    image: xxxxx/result:latest
    ports:
      - 5001:80
    depends_on:
      - db
    networks:
      - instavote

  worker:
    image: xxxxx/worker:latest
    depends_on:
      - redis
      - db
    networks:
      - instavote
```

***NOTE**: Be sure to replace **xxxxx** with your Docker Hub ID.*

As usual, launch it again with **`docker compose`**:

```
docker compose up -d
```

```
docker compose ps
```

You can find the complete Docker Compose file here.

Additional Docker Compose Commands to Explore

This command will list containers and the images used to create them.

```
docker compose images
```

The following commands will list logs of running services. The first will list all the logs of all the services. The second command demonstrates that you can specify a service from the `docker-compose.yaml` as an option to only show the logs for that service, in this example **`worker`**:

```
docker compose logs
```

```
docker compose logs worker
```

The following command pulls an image associated with a service defined in a `docker-compose.yml` or `docker-stack.yml` file, but does not start containers based on those images.

```
docker compose pull
```

The following command will execute a command inside a service. The following example specifically runs the **`ps`** command inside the **`vote`** service:

```
docker compose exec vote ps
```

The following command stops running containers without removing them. They can be started again with **`docker compose start`**:

```
docker compose stop
```

The following command stops and removes containers and networks:

```
docker compose down
```

If you run **`docker compose logs`** after running **`docker compose down`**, nothing will be displayed because it removes everything. You can get started again with **`docker compose up -d`**.

Finishing Up

Push the changes you have made so far to the repository:

```
git add docker-compose.yaml
```

```
git commit -am "adding docker compose spec"
```

```
git push origin feature/monopipe
```

Integrate Docker Compose with Dockerfiles

So far, you have been using pre-baked images to deploy to development. You can also have Docker Compose build the images by calling the respective Dockerfiles. Below is the code which will allow you to do this. Replace **xxxxx** with your DockerHub user ID.

```
volumes:
  db-data:

networks:
  instavote:
    driver: bridge

services:
  vote:
    image: xxxxx/vote:latest
    build: ./vote
    ports:
      - 5000:80
    depends_on:
      - redis
    networks:
      - instavote

  redis:
    image: redis:alpine
    networks:
      - instavote

  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: "postgres"
      POSTGRES_PASSWORD: "postgres"
    volumes:
      - "db-data:/var/lib/postgresql/data"
      - "./healthchecks:/healthchecks"
    healthcheck:
      test: /healthchecks/postgres.sh
      interval: "5s"
    networks:
      - instavote

  result:
    image: xxxxx/result:latest
    build: ./result
```

```
    ports:
      - 5001:80
    depends_on:
      - db
    networks:
      - instavote

  worker:
    image: xxxxx/worker:latest
    build: ./worker
    depends_on:
      - redis
      - db
    networks:
      - instavote
```

You can access the Docker Compose spec with Dockerfile build integration here.

You can build and deploy using the following commands:

`docker compose config`

`docker compose build`

`docker compose build` (observe caching)

`docker compose up -d`

`docker compose ps`

Finally, be sure to run:

`docker compose down`

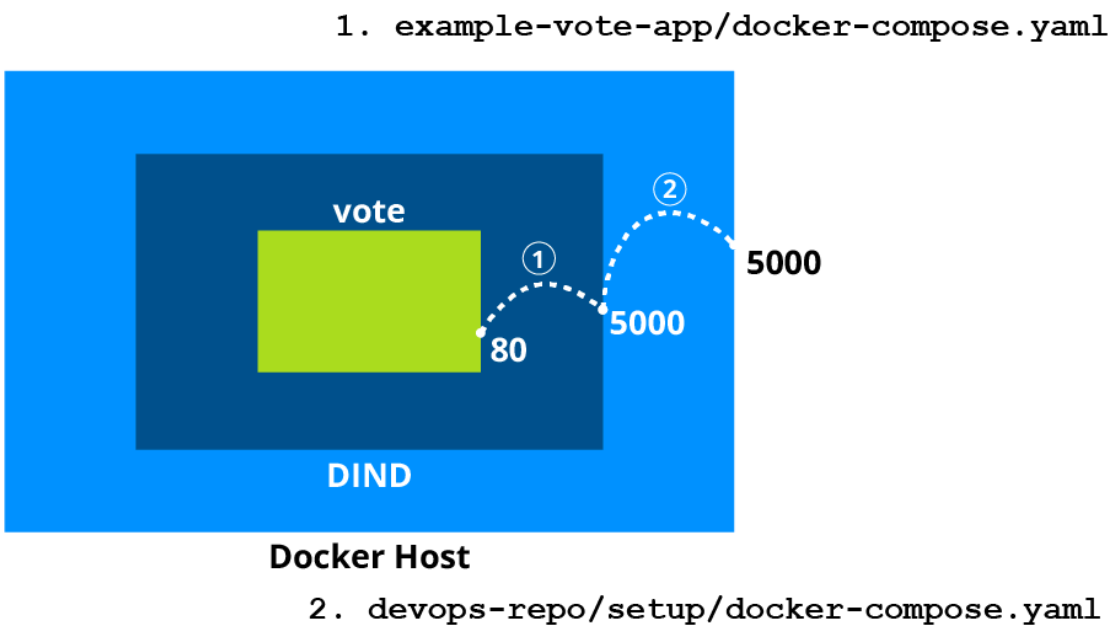
If you forget to do this, it will cause issues when you restart Jenkins and the DinD containers. Restart Jenkins and DinD by running the following from inside the `devops-repo/setup/` directory:

`docker compose up -d`

You are now ready to move on to the next section.

Add New Port Mapping

You will have to define an additional port mapping from inside `devops-repo/setup/docker-compose.yaml` and redeploy it. The following diagram depicts the two level port mappings:



Edit the **compose** spec used to set up Jenkins and DIND server by adding new port mappings. Be sure to remove any previous containers consuming these ports before you do so. Make sure the following is in **devops-repo/setup/docker-compose.yaml**:

```
docker:
  image: docker:dind
  ports:
    - 2376:2376
    - 5000:5000
    - 5001:5001
  environment:
    - DOCKER_TLS_CERTDIR=/certs
```

You can access the **docker-compose.yaml** file here.

After making this change, redeploy the setup using **docker compose up -d** from the setup directory. It will not disturb or recreate Jenkins, but it will relaunch the Docker DIND environment with new port mappings. Take care of any port conflicts that may exist while you bring up this environment.

Exercise: Create a Consolidated Pipeline for Instavote Stack

You have created one Jenkinsfile per application, matching **vote**, **worker**, and **result** apps.

Consolidating all those stages and building one single pipeline for the **`instavote`** app is called a **Mono Pipe**. As an exercise, use the following steps as a loose reference while attempting to complete this exercise on your own:

- Copy over **`worker/Jenkinsfile`** to the top level directory, i.e. **`example-voting-app/Jenkinsfile`.**
- Rename stages to identify with the application name. i.e. **`worker`**.
- Bring in just the stages from **`result/Jenkinsfile`** and **`vote/Jenkinsfile`**.
- Commit the changes to the repository branch created earlier, i.e. **`feature/monopipe`**.
- Create a new Multi Branch Pipeline job.

Once you validate the pipeline, add the “deploy to dev” stage.

Add “Deploy to Dev” with Docker Compose

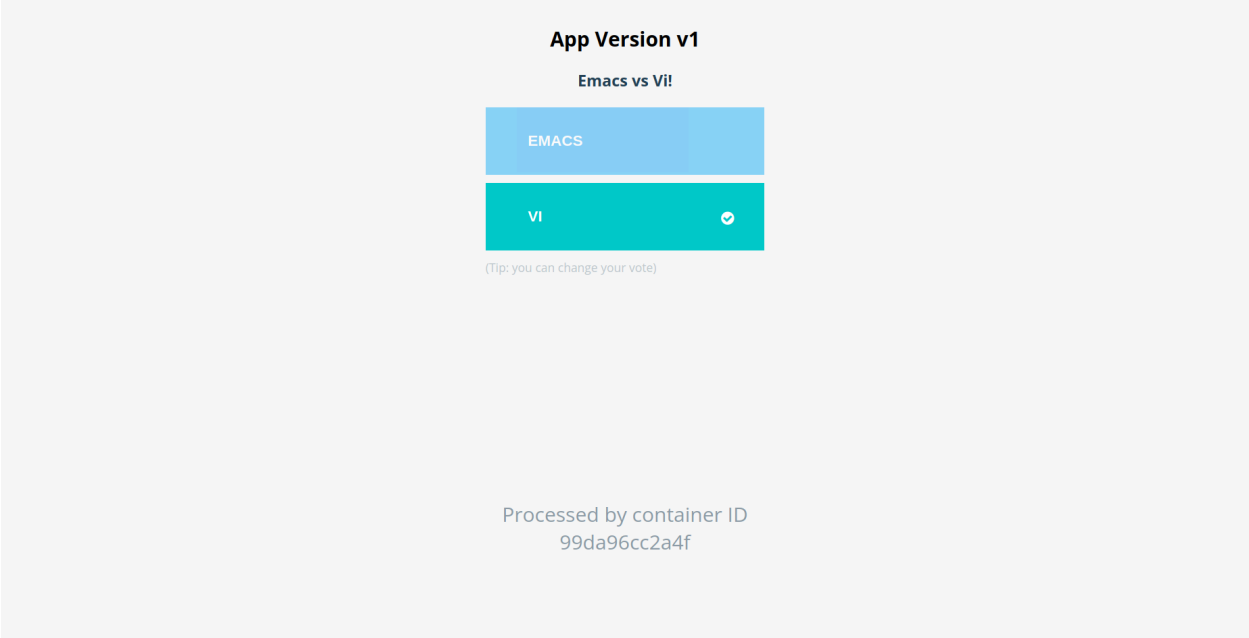
Add a “deploy to dev” stage to Jenkinsfile using the following stage snippet:

```
stage('deploy to dev'){
    agent any
    when{
      branch 'master'
    }
    steps{
      echo 'Deploy instavote app with docker compose'
      sh 'docker compose up -d'
    }
}
```

You can access the **“**deploy to dev” stage snippet here.

The entire Jenkinsfile for the Mono Pipe can be found here.

Be sure to push your changes to Github. After Jenkins runs the Mono Pipe job, you should be able to visit **`IPADDRESS:5000`**, for example **`localhost:5000`**, to see the vote application is up and running:



In this lab, we learned how to draft a `docker-compose.yaml` document and implemented a Mono Pipe with Jenkins. We also instructed Jenkins to automatically deploy our application. This concludes Lab 8.